

UNIVERSITY OF OKARA

Department of Computer Science

DATA STRUCTURES & ALGORITHMS

Lab Manual

LAB 14 | Binary Search Trees: Search, Insert, Delete and Balance

Week 14 of 16 • Duration: 3 hours (in-lab) + home practice

Prepared by

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

Applicable to: BSCS • BSIT • BSSE • BSAI • BSDS • BS Cyber Security and all programs offering DSA

Fall 2026 Edition

About This Lab Manual

This manual is part of a **16-week Data Structures & Algorithms (DSA) lab series**. It is designed so that a student starting with **zero programming knowledge** in Week 1 can confidently design, implement and analyse data structures by Week 16. Every lab builds on the previous one, so please do not skip labs.

Language policy: concepts are explained in a language-independent way (diagrams, pseudocode, dry runs). Full implementations are in **Python**, and every lab includes a **C++ Corner** so instructors teaching in C++ can use the same manual.

Every lab follows the same structure

Section	What you get
 Objectives & CLOs	What you will be able to do after this lab
 Motivation	Where this topic is used in real life, industry, research and the future (2026–2030)
 Concepts	Simple explanations with real examples, pseudocode and dry runs
 Code	Python implementation + C++ Corner
 Lab Tasks	Graded tasks:  Easy,  Medium,  Hard
 LeetCode	Real interview problems to practise
 Home Task &  Viva	Practice at home and prepare for oral questions
 Notebooks	Instructor notebook (complete code) and Student notebook (with TODO blanks and auto-check tests)

Lab 14 at a glance

Item	Details
Lab title	Binary Search Trees: Search, Insert, Delete and Balance
Week	14 of 16
Prerequisites	Lab 04 (Binary Search), Lab 13 (Binary Trees)
Tools	Google Colab / Jupyter / VS Code (Python 3); any C++ compiler for the C++ Corner
Files	Lab14_Instructor.ipynb , Lab14_Student.ipynb
Next lab	Lab 15: Heaps and Priority Queues

1. Learning Objectives

By the end of this lab, you will be able to:

1. State the **BST property** and check whether a tree satisfies it.
2. Implement **search, insert, minimum and maximum** recursively and iteratively.
3. Implement **deletion** for all three cases: leaf, one child, two children (inorder successor).
4. Explain why **inorder traversal of a BST gives sorted output**.
5. Analyse BST operations as **O(h)**: $O(\log n)$ when balanced, $O(n)$ when skewed.
6. Explain **balancing** with the AVL balance factor and rotations.
7. Solve range, k-th smallest, floor/ceiling and lowest-common-ancestor problems.

Course Learning Outcomes (CLO) mapping

CLO	Description	Taxonomy level
CLO-2	Implement binary search tree operations	Apply (C3)
CLO-3	Analyse BST performance and the need for balancing	Analyse (C4)

2. Motivation: Binary Search That Can Grow

Binary search on a sorted array is very fast ($O(\log n)$), but inserting a new element into a sorted array is slow ($O(n)$ shifting). A linked list inserts fast but searches slowly. A **Binary Search Tree (BST)** gives us the best of both: **search, insert and delete in $O(\log n)$** when the tree is balanced, while keeping data in sorted order.

Structure	Search	Insert	Delete	Sorted order?
Unsorted array	$O(n)$	$O(1)$	$O(n)$	No
Sorted array	$O(\log n)$	$O(n)$	$O(n)$	Yes
Linked list	$O(n)$	$O(1)$ at front	$O(n)$	No
Hash table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	No
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	Yes

2.1 BSTs in Daily Life

- **Guess-the-number game:** “Is it bigger or smaller than 50?” Each question moves you left or right, just like searching a BST.
- **A dictionary’s guide words:** decide whether your word comes before or after, and narrow down.
- **Leaderboards and rankings** that change constantly (game scores, PSL standings) need fast insert + sorted order.

2.2 BSTs in Industry

- **Ordered maps and sets:** C++ `std::map/std::set` and Java `TreeMap/TreeSet` are balanced BSTs (Red-Black trees).
- **Databases:** indexes are B-trees and B+ trees, multi-way generalisations of BSTs optimised for disks.
- **Operating systems:** the Linux Completely Fair Scheduler uses a Red-Black tree to pick the next process.
- **Range queries:** “all orders between Rs. 1,000 and Rs. 5,000”, “all students with CGPA 3.0–3.5” are fast on ordered trees.

2.3 BSTs in Research

- **Spatial data / GIS:** k-d trees (a BST that alternates between x and y coordinates) find nearest points on a map.
- **Machine learning:** k-d trees and ball trees speed up k-nearest-neighbour search.
- **Self-adjusting and learned index structures** are active database research topics.

2.4 Why It Matters in 2026–2030

Ordered data is still king

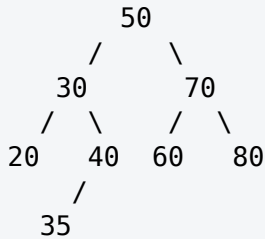
Hash tables are fast but forget order. Whenever you need **ordered** data with fast updates (time-series, leaderboards, ranges, nearest values), balanced trees are the answer. Understanding the BST is also the gateway to B-trees (databases), k-d trees (AI and GIS) and interval trees (scheduling).

3. Part A: The BST Property

A **Binary Search Tree** is a binary tree where, for **every** node:

- All keys in the **left subtree** are **smaller** than the node's key.
- All keys in the **right subtree** are **greater** than the node's key.
- Both subtrees are also BSTs. (In this lab, **no duplicate keys**.)

A valid BST



Inorder: 20 30 35 40 50 60 70 80 (sorted!)



A common trap

It is **not enough** that each node is bigger than its left child and smaller than its right child. Suppose 55 is placed as the right child of 40 in the tree above. Locally $40 < 55$ looks fine, but 55 is in the **left subtree of 50** while being greater than 50, so the tree is **not** a BST. Every key must respect **all** of its ancestors.



4. Part B: Search, Insert, Min and Max

4.1 Search

Compare the key with the current node: equal → found; smaller → go left; greater → go right; reach **None** → not present. At each step we discard one subtree, like binary search.

```

class Node:
    def __init__(self, key, left=None, right=None):
        self.key = key
        self.left = left
        self.right = right

def insert(root, key):
    """Insert key into the BST and return the (possibly new) root."""
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    elif key > root.key:
        root.right = insert(root.right, key)
    return root
    # duplicates are ignored

def build_bst(keys):
    root = None
    for k in keys:
        root = insert(root, k)
    return root

def inorder(root):
    return inorder(root.left) + [root.key] + inorder(root.right) if root else []

def search(root, key):
    # recursive
    if root is None or root.key == key:
        return root
    return search(root.left, key) if key < root.key else search(root.right, key)

def search_iter(root, key):
    # iterative: O(1) extra space
    while root and root.key != key:
        root = root.left if key < root.key else root.right
    return root

root = build_bst([50, 30, 70, 20, 40, 60, 80, 35])
print("inorder:", inorder(root))
print("search 35:", search(root, 35) is not None, "| search 65:", search_iter(root, 65) is
not None)

```

4.2 Insert

Search for the key; the place where the search falls off the tree (**None**) is exactly where the new node belongs. New keys are always inserted as **leaves**.

Insert	Path	Placed as
50	-	root
30	30 < 50 → left	left of 50
70	70 > 50 → right	right of 50
40	40 < 50 → left, 40 > 30 → right	right of 30
35	35 < 50, > 30, < 40	left of 40

4.3 Minimum and Maximum

```
def find_min(root):                                # keep going left
    while root.left:
        root = root.left
    return root.key

def find_max(root):                                # keep going right
    while root.right:
        root = root.right
    return root.key

print("min:", find_min(root), "| max:", find_max(root))
```

5. Part C: Deletion (Three Cases)

Case	What to do	Example (tree above)
1. Leaf node	Simply remove it	Delete 20
2. One child	Replace the node with its only child	Delete 40 → 35 takes its place
3. Two children	Replace the key with its inorder successor (smallest key in the right subtree), then delete the successor from the right subtree	Delete 50 → 60 becomes the root

Case 3 step by step

Delete 50 (two children):



successor = smallest in right subtree = 60

```

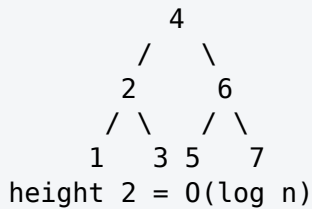
def delete(root, key):
    if root is None:
        return None
    if key < root.key:
        root.left = delete(root.left, key)
    elif key > root.key:
        root.right = delete(root.right, key)
    else:
        # found the node to delete
        # case 1 or 2
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        # case 2
        succ = root.right
        # case 3: inorder successor
        while succ.left:
            succ = succ.left
        root.key = succ.key
        root.right = delete(root.right, succ.key)
    return root

r = build_bst([50, 30, 70, 20, 40, 60, 80, 35])
for k in [20, 40, 50]:
    r = delete(r, k)
    print(f"after deleting {k}: inorder = {inorder(r)}, root = {r.key}")
    
```

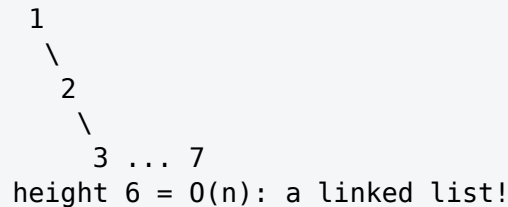
6. Part D: Balanced vs Skewed Trees

All BST operations walk one path from the root, so they cost **$O(h)$** where h is the height. The height depends on the **order of insertion**:

Insert 4, 2, 6, 1, 3, 5, 7



Insert 1, 2, 3, 4, 5, 6, 7 (sorted!)



```

import random, sys
sys.setrecursionlimit(10_000)

def height(node):
    return -1 if node is None else 1 + max(height(node.left),
    height(node.right))

n = 1000
keys = list(range(n))
print("sorted insert height  :", height(build_bst(keys)))
random.shuffle(keys)
print("random insert height  :", height(build_bst(keys)))
import math
print("best possible (log2 n):", math.floor(math.log2(n)))
  
```

6.1 Self-Balancing Trees: AVL Rotations

An **AVL tree** keeps the **balance factor** = height(left) – height(right) in $\{-1, 0, +1\}$ at every node. After an insert or delete, if a node becomes unbalanced, a **rotation** fixes it in $O(1)$. This guarantees height $O(\log n)$.

Right rotation

Right rotation at z (left-left case):



```

def rotate_right(z):
    y = z.left
    z.left = y.right      # T3 moves under z
    y.right = z
    return y              # y is the new root of this subtree

def rotate_left(z):
    y = z.right
    z.right = y.left
    y.left = z
    return y

skewed = build_bst([30, 20, 10])      # a left-left chain
balanced = rotate_right(skewed)
print("root:", balanced.key, "| left:", balanced.left.key, "| right:",
      balanced.right.key)

```

Imbalance case	Fix
Left-Left	Right rotation
Right-Right	Left rotation
Left-Right	Left rotation on child, then right rotation
Right-Left	Right rotation on child, then left rotation

Other balanced trees: **Red-Black trees** (used in C++ `map`, Java `TreeMap`, Linux), **B-trees** (databases), **Splay trees** and **Treaps**.

7. C++ Corner

C++

```
#include <iostream>
#include <map>
#include <set>
using namespace std;

struct Node { int key; Node *left = nullptr, *right = nullptr; Node(int k) :
key(k) {} };

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->key) root->left = insert(root->left, key);
    else if (key > root->key) root->right = insert(root->right, key);
    return root;
}

bool search(Node* root, int key) {
    while (root && root->key != key) root = key < root->key ? root->left : root->right;
    return root != nullptr;
}

int main() {
    Node* root = nullptr;
    for (int k : {50, 30, 70, 20, 40, 60, 80}) root = insert(root, k);
    cout << search(root, 60) << search(root, 65) << endl;    // 10

    set<int> s = {50, 30, 70, 20};           // balanced BST (Red-Black tree)
    auto it = s.lower_bound(35);             // smallest key >= 35 -> 50
    cout << *it << endl;
    map<string, double> cgpa = {"Sara", 3.9}, {"Ali", 3.4}};
    for (auto& [name, g] : cgpa) cout << name << " " << g << endl;    // sorted by
name
}
```

8. Common Mistakes

Mistake	Effect	Fix
Checking only parent–child order when validating	Accepts invalid BSTs	Pass (low, high) bounds down the recursion
Forgetting <code>root.left = insert(...)</code>	New node is created but never attached	Always assign the returned subtree
Deleting a two-child node by removing it	Tree breaks apart	Copy the successor's key, delete the successor
Inserting sorted data into a plain BST	Height n , $O(n)$ operations	Shuffle, or use a balanced tree

Mistake	Effect	Fix
Allowing duplicates without a rule	Search/delete become inconsistent	Ignore duplicates or store a count

9. Lab Tasks

Solve all tasks in `Lab14_Student.ipynb`. Each task has a **test cell**: when it prints  your solution is correct. For each task, **write pseudocode first**, then code, then test with your own inputs including edge cases. Difficulty:  Easy  Medium  Hard.

Task 1 Validate a BST

Write `is_bst(root)` that checks the **full** BST property. Pass down the allowed range (`low`, `high`) for each subtree.

The notebook provides `Node`, `insert`, `build_bst` and `inorder`.

```
Tree 50 → (30 → right 55), 70 → False (55 is in the left subtree of 50)
```

Task 2 k-th Smallest Mark

Marks are stored in a BST. Write `kth_smallest(root, k)` using an inorder traversal that **stops early** once the k-th element is reached (do not build the whole list).

```
BST of [50, 30, 70, 20, 40, 60, 80], k = 3 → 40
```

Task 3 Range Query: Students Between Two CGPAs

Keys are CGPA × 100 (integers). Write `range_keys(root, low, high)` that returns all keys with $low \leq key \leq high$ in sorted order. **Prune**: do not visit the left subtree if `root.key < low`, nor the right subtree if `root.key > high`.

```
keys [310, 250, 365, 290, 330, 390, 280], range [280, 340] → [280, 290, 310, 330]
```

Task 4 Floor and Ceiling

`floor(root, x)` = largest key ≤ x; `ceiling(root, x)` = smallest key ≥ x (return `None` if none). Solve both in O(h) **without** traversing the whole tree.

Use: a bus timetable stored as minutes after midnight; find the last bus before you arrive and the next bus after you arrive.

```
times [480, 540, 600, 720, 840], x = 610 → floor 600, ceiling 720
```

Task 5 🟡 Lowest Common Ancestor in a BST

The **lowest common ancestor (LCA)** of two keys is the deepest node that has both keys in its subtree (a node can be its own ancestor). In a BST, use the ordering: if both keys are smaller go left, if both are bigger go right, otherwise the current node is the answer. $O(h)$.

```
BST [6, 2, 8, 0, 4, 7, 9, 3, 5]: lca(2, 8) → 6, lca(2, 4) → 2, lca(3, 5) → 4
```

Task 6 🔴 Sorted Array to Balanced BST

Given sorted roll numbers, build a **height-balanced** BST: make the middle element the root, then recursively build the left half and right half. Return the root.

Verify that the height is $\lfloor \log_2 n \rfloor$ for $n = 1,023$ and that inorder gives back the array.

```
[1, 2, 3, 4, 5, 6, 7] → root 4, left subtree root 2, right subtree root 6
```

Task 7 🔴 A Complete BST Map Class

Build a class **BSTMap** storing **key** → **value** pairs with methods **put(key, value)** (insert or update), **get(key)** (or **None**), **remove(key)**, **__len__**, **items()** (sorted list of pairs) and **height()**.

Use it as a sorted phone book. Internally, store **key** and **value** in each node and reuse the deletion logic from Part C.

```
put("Sara", "0300"); put("Ali", "0321"); items() → [("Ali", "0321"), ("Sara", "0300")]
```



10. LeetCode Practice

Solve these on **leetcode.com** and upload screenshots of the “Accepted” result to your GitHub **DSA-Labs/Lab14** folder.

#	Problem	Level	Concept practised
700	Search in a Binary Search Tree	Easy	BST search
701	Insert into a Binary Search Tree	Medium	BST insert
450	Delete Node in a BST	Medium	Three deletion cases
98	Validate Binary Search Tree	Medium	Bounds (Task 1)
230	Kth Smallest Element in a BST	Medium	Inorder (Task 2)

#	Problem	Level	Concept practised
938	Range Sum of BST	Easy	Pruning (Task 3)
235	Lowest Common Ancestor of a BST	Medium	Ordering (Task 5)
108	Convert Sorted Array to Binary Search Tree	Easy	Divide & conquer (Task 6)



11. Home Task

1. Finish all tasks and push `Lab14_Student.ipynb` to GitHub.
2. Implement `inorder_successor(root, key)` and `inorder_predecessor(root, key)` in $O(h)$.
3. Implement a full **AVL tree insert** that stores heights in nodes and applies the four rotation cases. Insert 1 to 1,000 in sorted order and confirm that the height stays near $\log_2 n$.
4. **Mini-project: Live Leaderboard.** Players get scores over time. Store (score, name) in your `BSTMap` and support: add/update score, top-10 players (reverse inorder), rank of a player, and players with scores in a range.
5. Solve at least 3 LeetCode problems from the list.

? 12. Viva Questions

1. State the BST property.
2. Why does inorder traversal of a BST give sorted output?
3. Explain how to search for a key in a BST. What is its complexity?
4. Where is a new key always inserted in a BST?
5. Explain the three cases of deletion.
6. What is an inorder successor? Why is it used in deletion?
7. When does a BST become a linked list? What is the complexity then?
8. What is the balance factor in an AVL tree?
9. Name two real systems that use balanced BSTs.
10. Compare a BST with a hash table: when would you choose each?



13. Assessment Rubric

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs work (0–1)
Correctness	All tasks pass all tests	Most tasks correct	Some tasks correct	Mostly incorrect
Logic & pseudocode	Clear pseudocode and dry run for every task	Pseudocode for most tasks	Partial pseudocode	None
Code quality	Meaningful names, functions, comments	Mostly clean	Hard to read	Copied or unreadable
Complexity & edge cases	States Big-O and handles all edge cases	Some edge cases / Big-O	Only basic cases	Not considered
Viva	Explains every line confidently	Explains most parts	Basic understanding	Cannot explain



Lab Rules

- Bring your own laptop with Colab or Jupyter working.
- Arrive on time; late arrival may result in no attendance for the lab.
- Copying another student's code or pasting AI output without understanding it will be awarded zero. You must be able to explain every line in the viva.



14. References and Further Learning

- BST and AVL animations: visualgo.net/en/bst
- C++ `std::map` / Java `TreeMap` documentation (Red-Black trees).
- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, **Introduction to Algorithms** (CLRS), 4th ed., MIT Press, 2022.
- M. T. Goodrich, R. Tamassia, M. H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley.
- MIT OpenCourseWare 6.006 Introduction to Algorithms; Harvard CS50x (free online courses).
- Visualise code step by step: pythontutor.com; animated data structures: visualgo.net.



About the Author

Muhammad Ramzan

MPhil Scholar (Artificial Intelligence), PUCIT, University of the Punjab, Lahore

BS Computer Science, University of Okara

Lab Engineer, Department of Computer Science, University of Okara

GitHub: github.com/RamzanPUCIT

LinkedIn: linkedin.com/in/ramzan-ai

ResearchGate: researchgate.net/profile/Muhammad-Ramzan-105

© 2026 Muhammad Ramzan. Prepared for the Department of Computer Science, University of Okara.